PACKAGE 11 / VOLUME 10

Math and reference

Equations, conventions and sources

Worked quantities · Evidence boundaries · Primary readings

A companion for checking signs, units, guarantees and the exact source behind a technical claim.

September 28, 2026
Graduate self-study specialization

Contents

1. Memory and architecture arithmetic . . . 3

2. Timing signs and units . . . 3

3. Normalized inverter-chain estimate . . . 4

4. Power, energy and constrained choices . . . 4

5. SECDED bit convention and campaign size . . . 4

6. Paired performance analysis . . . 5

7. Address validation and failure boundaries . . . 5

8. Evidence vocabulary . . . 5

9. Reading strategy and version discipline . . . 6

Primary-source directory . . . 7

Original program reading concordance . . . 11

Equations, conventions, evidence boundaries and primary sources

This is a companion to the handbook, not an alternative to the derivations. All numerical fixtures below are original teaching examples. Units and exclusions are part of each formula. A unit test validates the implemented arithmetic against its test contract; it does not validate a physical model for a particular device.

1. Memory and architecture arithmetic

For a cache with S sets, W ways and B bytes per line, data capacity is S × W × B bytes. For byte address a, block = floor(a/B), set = block mod S, and tag = floor(block/S). This describes the supplied simple modulo-indexed cache, not every hardware indexing function. Two configurations with the same number of sets but different ways also have different capacity.

The supplied serial model uses total cycles = N × h + M × p, where N is requests, h is lookup cost per request, M is misses and p is **additional** miss penalty after lookup. For N=100, h=2, M=10 and p=30, total modeled cycles are 200+300=500. If a supplied miss cost already includes lookup, the formula must change to avoid counting it twice. The model omits overlap, bandwidth saturation, prefetching, writebacks and real engine work. [S03, S06]

For an overlap upper-bound argument, keep bandwidth and compute limits separate. Arithmetic intensity is operations divided by transferred bytes under a stated traffic scope; a bandwidth bound is intensity × bandwidth. A minimum of a compute peak and a bandwidth-derived limit is an upper-bound model, not a guarantee that a real application reaches it. Latency, dependencies and scheduling can leave the actual result lower. [S06]

For the fixed-latency scheduler, issue at cycle t with latency L produces completion at t+L. Each cycle processes completion, then at most one in-order retirement, then issue. Its four-instruction fixture has identical final retirement cycle 7 under both issue policies despite earlier independent execution out of order. This is a useful negative result: local overlap does not remove a retirement bottleneck. [S04, S05]

2. Timing signs and units

Let positive skew k = capture-clock arrival minus launch-clock arrival. Let T be the clock period, qmax/qmin clock-to-Q bounds, dmax/dmin combinational bounds, s setup time, h hold time, us setup uncertainty and uh hold uncertainty. Under this simple single-cycle same-edge convention:

```
setup slack = T + k - us - s - qmax - dmax
hold slack  = qmin + dmin - k - h - uh
```

Use one consistent time unit. With T=2.00 ns, k=0.10, us=0.08, s=0.15, qmax=0.12 and dmax=1.50, setup slack is +0.25 ns. With qmin=0.05 ns, dmin=0.08, h=0.06 and uh=0.03, hold slack is −0.06 ns. Positive capture skew helps setup and hurts hold. Reducing the period to 1.60 ns changes setup to −0.15 ns while leaving this hold expression unchanged.

These are instructional equations, not a substitute for a timing engine’s exact edge, derating, uncertainty and path conventions. Generated clocks, multicycle paths, clock reconvergence and other effects need their own contract. A false-path declaration removes an analysis requirement; it does not repair a physical failure. Do not exempt a synchronous reset merely because the word “reset” appears in its name. [S08]

3. Normalized inverter-chain estimate

For total load ratio H, N identical-effort stages and normalized parasitic p, the illustrative delay is D = N × (H^(1/N) + p). Relative input sizes are 1, f, f², … where f = H^(1/N). With H=64 and p=1, N=1,2,3,4 gives approximately 65, 18, 15 and 15.314 normalized units. For three stages, f≈4 and sizes≈1,4,16.

The expression assumes a simplified effort model, ignores discrete cell options and wiring, and does not establish characterized delay. Stage parity determines inversion: three inverters invert; four preserve polarity. A parity constraint can eliminate the apparent unconstrained winner. Floating-point tests should compare these derived values with an appropriate tolerance rather than demanding identical decimal representations. Physical-flow materials in S07–S10 are the next evidence tier.

4. Power, energy and constrained choices

Using alpha as charging events per cycle, switched dynamic power is alpha × C × V² × f in watts when C is farads, V volts and f hertz. The convention matters: a different definition of transitions can introduce a different factor. For alpha=.2, C=10 pF, V=1 V and f=100 MHz, the result is .0002 W = .2 mW. Reducing V to .8 V gives .128 mW **under fixed C, activity and frequency**; it does not establish that timing remains feasible. The expression excludes leakage and other system costs. [S07, S09]

Energy per completed item is power divided by throughput when both refer to the same operating interval and system boundary. At 2 mW and one million items/s, energy is 2 nJ/item. At 3 mW and two million items/s, it is 1.5 nJ/item. A lower ratio can coexist with higher total power and may violate a hard power limit.

A candidate q strictly dominates p when every minimized objective of q is no worse than p and at least one is strictly better. Identical vectors do not strictly dominate each other. In the synthetic latency/power/area example A=(10,40,20), B=(8,50,24), C=(12,45,22), D=(9,35,25), A dominates C and the frontier is A,B,D. Filter hard constraints before choosing among frontier points. A 42 W ceiling and area limit 24 admit A, whereas area limit 26 also admits D, which has lower latency. These numbers are not device specifications.

5. SECDED bit convention and campaign size

The reference uses an extended Hamming(8,4) code. Positions 1,2,4 hold parity; positions 3,5,6,7 hold payload bits from least to most significant; position 8 holds overall parity. Position 1 maps to the least-significant bit of the encoded byte. Even parity is used. Payload decimal 10 encodes as hexadecimal D2.

For each received word, compute the Hamming syndrome over positions 1–7 and overall parity over all eight bits. Zero syndrome/even overall parity means no detected error, not a universal guarantee of no corruption. Nonzero syndrome/odd parity selects a single-bit correction in positions 1–7; zero syndrome/odd parity selects the overall-parity bit; nonzero syndrome/even parity identifies a detected double error under the stated fault model. Do not release a payload for the detected-double status.

There are 16 clean codewords, 16×8=128 single-bit injections and 16×C(8,2)=448 double-bit injections. This exhaustive campaign validates those finite classes. Triple-bit errors can be miscorrected: starting from zero and flipping positions 1,2,3 can produce a decoded payload of one after the decoder treats the syndrome/parity as a single-error case. “Single-error correction, double-error detection” does not promise arbitrary fault correction. Linux EDAC provides wider reporting context, not this exact code’s specification. [S15]

6. Paired performance analysis

For paired positive times b_i and c_i, define speedup r_i=b_i/c_i. The geometric mean speedup is exp(mean(log(b_i)−log(c_i))). It weights ratios symmetrically in log space; it does not equal total baseline time divided by total candidate time in general. Choose the aggregate because it matches the research question, not because it gives a larger number.

For baseline [10,12,8,20] ms and candidate [8,9,7,18] ms, the geometric mean is approximately 1.206145. A speedup of 1.25 corresponds to candidate time .8 of baseline, a 20% time reduction, and a 25% throughput increase only when throughput is the inverse of that time under an otherwise unchanged boundary.

The provided bootstrap samples whole paired indices with replacement, computes the same statistic per resample, sorts the bootstrap estimates and interpolates quantiles at index (n−1)×p. It uses 2,000 resamples and seed 20260928 in the synthetic demo. The resulting nominal 95% percentile interval is approximately [1.126872, 1.290994] for that fixture. Four synthetic pairs are a mechanics demonstration, not evidence for a real performance claim. Coverage depends on the experimental design and distribution; the numerical label “95%” is not a guarantee. [S18]

7. Address validation and failure boundaries

For buffer capacity C, require nonnegative offset o, positive length L and the declared alignment. Check o≤C and L≤C−o rather than relying on o+L≤C in a fixed-width integer type that can overflow. Match owner and active generation, copy the validated descriptor into protected model state, and reject stale completions after reset. The local example uses C=4096, alignment 16 and owner 7; o=4080,L=16 passes, while L=32 fails.

These are local-model checks. Real DMA also involves mapping, access permissions, lifetime, coherency and device-specific contracts. No code here performs physical transfers or configures an IOMMU. A reset epoch is not a rollback of writes already made by real hardware. [S16]

A simple steady-state thermal estimate Tj≈Ta+P×theta uses degrees Celsius for temperatures, watts for P and degrees Celsius per watt for theta. A synthetic Ta=30, P=8 and theta=5 produces 70°C. This is not a safe limit for any device and ignores transient and spatial effects. Preserve platform thermal protections and obtain actual limits from the relevant device documentation. [S17]

8. Evidence vocabulary

Architectural state: programmer-visible state that retirement must preserve despite internal overlap. **Coherence:** rules for consistent ownership and values for a location. **Consistency:** constraints on observable memory-operation order. **Happens-before:** the language-level relation needed to justify communicating non-atomic state. **Transient state:** an in-progress protocol condition whose messages and acknowledgments have not all completed.

Setup/hold slack: margin under the stated late/early timing requirement. **PPA:** power, performance and area, with the technology, stage and estimation assumptions attached. **Corner:** a declared set of process, voltage and temperature conditions, not a guarantee covering every unmodeled variation. **CDC:** a transfer between clock domains whose protocol and metastability concerns must both be considered. **MTBF:** a reliability estimate that requires device and implementation assumptions beyond digital simulation.

Assumption: a restriction placed on the environment or model. **Assertion:** a property being checked. **Cover:** a reachability target used to show interesting behavior is possible. **Vacuity:** a pass caused by conditions that

prevent the relevant behavior from occurring. **Counterexample:** a trace demonstrating a violated property under its modeled assumptions. **Ablation:** removal of a claimed mechanism. **Sensitivity study:** variation of a parameter or assumption. **Independent reproduction:** another person rerunning the central result from the released instructions and inputs.

9. Reading strategy and version discipline

Consult the primary source for the specific claim: the language standard for C++ ordering, the named ISA for allowed executions, the tool documentation for commands, and the actual library/device documentation for physical assumptions. Record exact versions when a claim depends on them. A stable historical paper and a moving “latest” documentation page have different reproducibility needs.

The S-numbered directory below was checked on September 28, 2026. The package does not bundle third-party textbooks, toolchains or restricted libraries. The worked examples, code and instructional wording are original teaching material. A source link is assigned reading, not evidence that an external tool was run here.

Primary-source directory

S01 — C++ working draft: order and consistency of atomics

Language-level release/acquire synchronization. Live working draft; record a dated snapshot for a thesis. The bundled example targets C++17, not a claim to implement every feature in this live draft.

Source: https://eel.is/c++draft/atomics.order

Accessed September 28, 2026.

S02 — RISC-V unprivileged ISA: RVWMO, v20260120

A named hardware memory model for litmus reasoning. Use this version rather than a generic description of caches.

Source: https://docs.riscv.org/reference/isa/v20260120/unpriv/rvwmo.html

Accessed September 28, 2026.

S03 — gem5 documentation: Ruby

Coherence and memory-system modeling. A route to richer protocol studies; not validation of the package's atomic MSI model.

Source: https://www.gem5.org/documentation/general_docs/ruby/

Accessed September 28, 2026.

S04 — gem5 documentation: O3CPU

A detailed out-of-order CPU model. The package's fixed-latency scheduler is intentionally much smaller.

Source: https://www.gem5.org/documentation/general_docs/cpu_models/O3CPU

Accessed September 28, 2026.

S05 — MIT OpenCourseWare: 6.823 Computer System Architecture, Fall 2005

Historical architecture course notes for speculation, scheduling and precise state. Conceptual reading, not current product specifications.

Source: https://ocw.mit.edu/courses/6-823-computer-system-architecture-fall-2005/pages/lecture-notes/

Accessed September 28, 2026.

S06 — Williams, Waterman and Patterson: Roofline report, UCB/EECS-2008-134

Original performance-model report. Helps distinguish an upper bound and bottleneck model from an end-to-end measurement.

Source: https://www2.eecs.berkeley.edu/Pubs/TechRpts/2008/EECS-2008-134.html

Accessed September 28, 2026.

S07 — Cornell ECE 5745: Complex Digital ASIC Design course portal

Starting point for an educational ASIC flow and course materials. Choose and record a specific offering; this link alone does not pin its tools or libraries.

Source: https://www.csl.cornell.edu/courses/ece5745/

Accessed September 28, 2026.

S08 — OpenSTA documentation

Timing analysis, constraints and reporting. Pin a release or commit before relying on exact commands.

Source: https://openroad.readthedocs.io/en/latest/main/src/sta/README.html

Accessed September 28, 2026.

S09 — OpenROAD Flow Scripts: Flow Tutorial

Flow stages and example configuration. The bundled physical-flow entry points are learner-run inputs, not results.

Source: https://openroad-flow-scripts.readthedocs.io/en/latest/tutorials/FlowTutorial.html

Accessed September 28, 2026.

S10 — OpenROAD Flow Scripts: Adding New Designs

Design integration and configuration. Check the version-specific environment, platform and paths.

Source: https://openroad-flow-scripts.readthedocs.io/en/latest/user/AddingNewDesign.html

Accessed September 28, 2026.

S11 — YosysHQ SBY: formal extensions to Verilog

Assertions, assumptions, covers and past-state reasoning. Separate bounded checks from completed proofs.

Source: https://yosyshq.readthedocs.io/projects/sby/en/latest/verilog.html

Accessed September 28, 2026.

S12 — YosysHQ SBY: configuration file reference

Tasks, proof modes, depth and engines. A solver or engine timeout must remain UNKNOWN/failed-to-complete, not PASS.

Source: https://yosyshq.readthedocs.io/projects/sby/en/latest/reference.html

Accessed September 28, 2026.

S13 — AMD UG949: Multi-Bit CDC

Vendor guidance for coherent multi-bit clock-domain transfers. The Python mailbox demonstrates a protocol, not analog metastability behavior.

Source: https://docs.amd.com/r/en-US/ug949-vivado-design-methodology/Multi-Bit-CDC

Accessed September 28, 2026.

S14 — AMD UG949: Optimizing for MTBF

Synchronizer reliability and implementation considerations. Do not infer an MTBF number from digital simulation.

Source: https://docs.amd.com/r/en-US/ug949-vivado-design-methodology/Optimizing-for-MTBF

Accessed September 28, 2026.

S15 — Linux kernel documentation: EDAC

Error detection and correction interfaces and reporting. The 8-bit SECDED exercise is an educational code, not a model of a specific memory controller.

Source: https://docs.kernel.org/driver-api/edac.html

Accessed September 28, 2026.

S16 — Linux kernel documentation: DMA API HOWTO

DMA address and mapping contracts. The local descriptor checker does not perform real DMA or configure an IOMMU.

Source: https://docs.kernel.org/core-api/dma-api-howto.html

Accessed September 28, 2026.

S17 — Linux kernel documentation: thermal sysfs API

Thermal interfaces and policy concepts. Actual limits and protections are platform-specific and must not be bypassed.

Source: https://docs.kernel.org/driver-api/thermal/sysfs-api.html

Accessed September 28, 2026.

S18 — NIST/SEMATECH e-Handbook: paired observations

Why paired measurements require a paired analysis. The package's bootstrap routine is an original teaching implementation, not a quoted NIST procedure.

Source: https://www.itl.nist.gov/div898/handbook/prc/section3/prc311.htm

Accessed September 28, 2026.

S19 — Pro Git: Contributing to a Project

Patch preparation, review and contribution workflows. A local, reviewable integration can be educational evidence without guaranteed upstream acceptance.

Source: https://git-scm.com/book/en/v2/Distributed-Git-Contributing-to-a-Project

Accessed September 28, 2026.

S20 — CS2023: knowledge areas

A curriculum taxonomy for planning continued learning. Completing this package does not confer accreditation or a university degree.

Source: https://csed.acm.org/knowledge-areas/

Accessed September 28, 2026.

Original program reading concordance

The following R-numbered records are inherited from the supplied program manifest. They preserve the original assigned readings; they are not a claim that every external textbook, page or tool version was reverified for this expansion. Use S01–S20 for the checked primary-source links supporting the expansion, and record the exact edition you actually study.

R01 — CS2023 knowledge areas

ACM / IEEE-CS / AAAI. Breadth checklist, not a claim of accreditation.

Original manifest URL: https://csed.acm.org/knowledge-areas/

R02 — Computer Engineering Curricula 2016

ACM / IEEE Computer Society. Computer-engineering breadth and laboratory expectations; historical curriculum report.

Original manifest URL: https://ieeecs-media.computer.org/assets/pdf/ce2016-final-report.pdf

R05 — Pro Git

Scott Chacon and Ben Straub. Version control and collaboration.

Original manifest URL: https://git-scm.com/book/en/v2

R06 — 6.042J Mathematics for Computer Science

MIT OpenCourseWare. Proofs, graphs, counting and discrete probability.

Original manifest URL: https://ocw.mit.edu/courses/6-042j-mathematics-for-computer-science-spring-2015/

R09 — 18.05 Introduction to Probability and Statistics

MIT OpenCourseWare. Probability, inference and quantitative reasoning.

Original manifest URL:
https://ocw.mit.edu/courses/18-05-introduction-to-probability-and-statistics-spring-2022/

R11 — C++ Core Guidelines

C++ Core Guidelines editors. Resource safety, interfaces, classes and concurrency; guidance rather than the normative language standard.

Original manifest URL: https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines

R26 — Programming with C++ in Unreal Engine

Epic Games. Engine comparison after C++ foundations, not a beginner substitute.

Original manifest URL: https://dev.epicgames.com/documentation/en-us/unreal-engine/programming-with-cplusplus-in-unreal-engine

R30 — 6.002 Circuits and Electronics

MIT OpenCourseWare. Circuits, transistors and laboratory reasoning.

Original manifest URL: https://ocw.mit.edu/courses/6-002-circuits-and-electronics-spring-2007/

R33 — Yosys documentation

YosysHQ. Synthesis and technology mapping; stable installed release preferred over a development snapshot.

Original manifest URL: https://yosyshq.readthedocs.io/projects/yosys/en/latest/

R35 — RISC-V ISA specifications

RISC-V International. Instruction-set reference; record the exact ISA version and implemented extensions.

Original manifest URL: https://docs.riscv.org/reference/isa/

R38 — gem5 documentation

gem5 project. Architecture simulation; distinguish model outputs from measured hardware results.

Original manifest URL: https://www.gem5.org/documentation/

R39 — OpenROAD documentation

OpenROAD project. RTL-to-physical-design study; educational results are not production signoff.

Original manifest URL: https://openroad.readthedocs.io/en/latest/

R42 — Convex Optimization

Stephen Boyd and Lieven Vandenberghe. Graduate optimization text available from its authors; copyrighted, linked rather than redistributed.

Original manifest URL: https://web.stanford.edu/~boyd/cvxbook/

R43 — Roofline performance model, technical report

Samuel Williams, Andrew Waterman and David Patterson. Original 2008 performance-model reference; use current measured hardware limits for experiments.

Original manifest URL: https://www2.eecs.berkeley.edu/Pubs/TechRpts/2008/EECS-2008-134.html

R44 — Spatiotemporal reservoir resampling for real-time ray tracing with dynamic direct lighting

Benedikt Bitterli and coauthors. Original ReSTIR research, 2020; example reproduction target, not a claim to be the latest method.

Original manifest URL: https://research.nvidia.com/publication/2020-07_spatiotemporal-reservoir-resampling-real-time-ray-tracing-dynamic-direct

R48 — 6.823 Computer System Architecture

MIT OpenCourseWare. Graduate architecture foundations; historical course concepts supplemented with current specifications.

Original manifest URL: https://ocw.mit.edu/courses/6-823-computer-system-architecture-fall-2005/

R49 — ECE5745 Complex Digital ASIC Design

Christopher Batten / Cornell University. Graduate ASIC methodology, circuits, EDA and design-project expectations.

Original manifest URL: https://www.csl.cornell.edu/courses/ece5745/

R50 — SymbiYosys documentation

YosysHQ. Formal hardware verification, safety, liveness and cover properties.

Original manifest URL: https://yosyshq.readthedocs.io/projects/sby/en/latest/

R51 — Driver implementer API guide

Linux kernel contributors. Device ownership, I/O, DMA and driver interfaces; use disposable VMs or simulated devices for kernel experiments.

Original manifest URL: https://docs.kernel.org/driver-api/index.html

R55 — Author publication archive: position-based simulation and XPBD

Matthias Mueller and collaborators. Read the named 2016 XPBD paper and related course notes for the simulation reproduction; papers retain their own copyrights.

Original manifest URL: https://matthias-research.github.io/pages/publications/publications.html